Ayan Ghanchi

Social Media System:

Introduction:

The Social Media System is a web application that allows users to create accounts, post photos and videos, and interact with posts through likes and comments. The main users are people who want to share content and interact with others. The backend will use spring boot, spring data JPA, an H2 database, DTOs, and a layered architecture with controller, service, repository, and domain layers.

Design:

Resources

User

Represents a registered user.

Attributes:

- id: Long
- username: String
- email: String
- displayName: String

Post

Represents content created by a user.

Attributes:

- id: Long
- content: String
- mediaUrl: String
- createdAt: LocalDateTime
- author: User

Ayan Ghanchi

Interaction

Represents a like or comment on a post.

Attributes:

- id: Long
- type: InteractionType
- commentText: String
- createdAt: LocalDateTime
- user: User
- post: Post

InteractionType contains:

- LIKE
- COMMENT

DTOs

Each resource will use request and response DTOs.

User

UserRequestDTO: username, email, displayName

UserResponseDTO: id, username, email, displayName

Post

PostRequestDTO: authorId, content, mediaUrl

PostResponseDTO: id, authorId, authorUsername, content, mediaUrl, createdAt

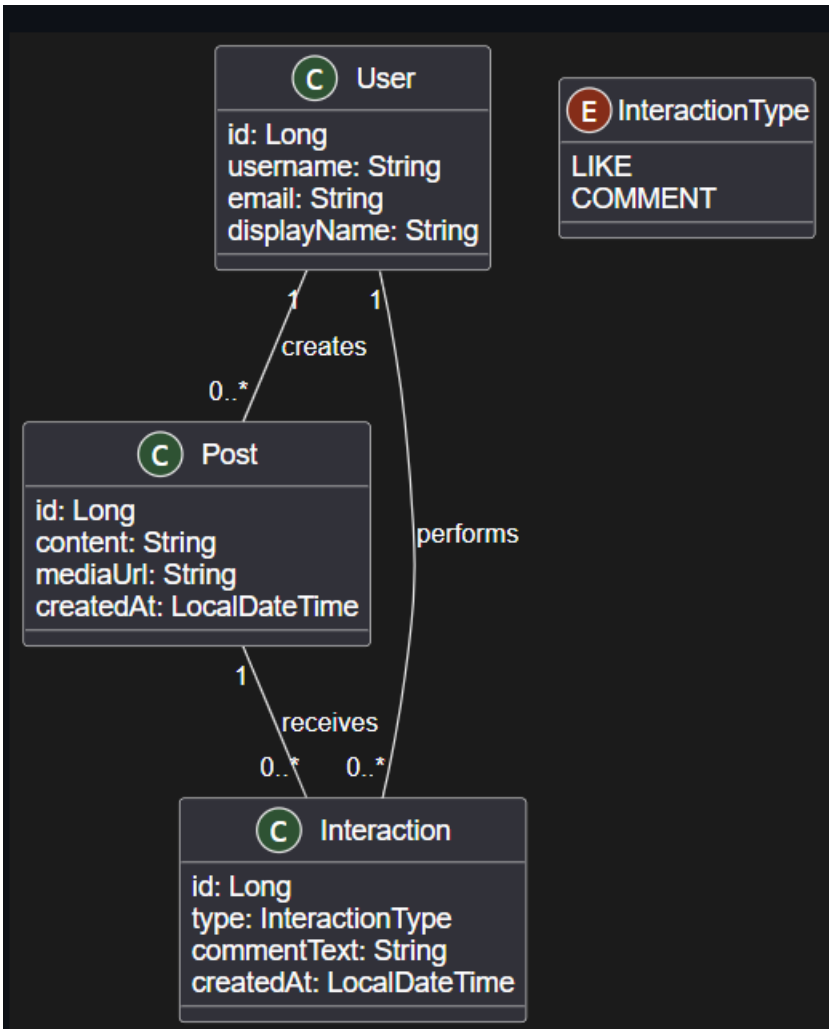
Interaction

InteractionRequestDTO: userId, postId, type, commentText

InteractionResponseDTO: id, userId, username, postId, type, commentText, createdAt

UML Class Diagram

Ayan Ghanchi



Relationships

- **User to Post:** One-to-Many
- **User to Interaction:** One-to-Many
- **Post to Interaction:** One-to-Many

Each post belongs to one user.

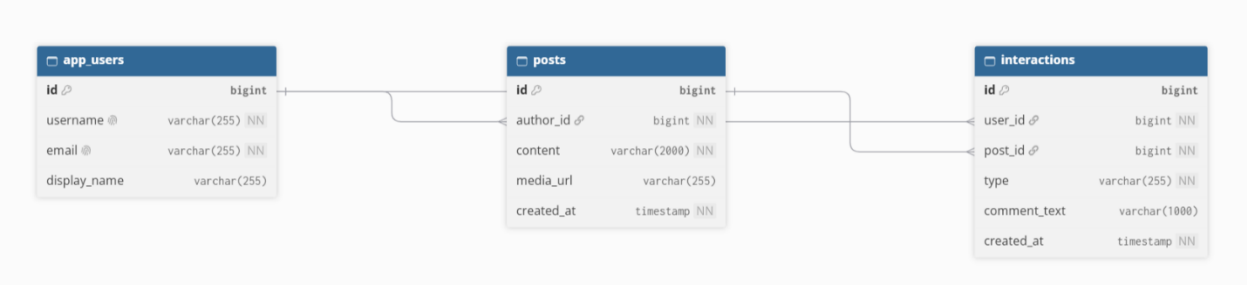
Each interaction belongs to one user and one post.

Collections will use FetchType.LAZY.

CascadeType.PERSIST can be used on one-to-many relationships.

ERD / Database Schema:

Ayan Ghanchi



Wireframes

SOCIAL MEDIA SYSTEM
User Profile **Username: johnsmith** **Display Name: John Smith**
User Posts **Post 1: Hello everyone!** **Post 2: My first social media post.**
Home | Profile | Posts

SOCIAL MEDIA SYSTEM
Post Details **Author: johnsmith** **Content: Hello everyone!**
Interactions **Likes** **Comments**
Like | Comment | Back

Ayan Ghanchi

REST Endpoints:

Users

Method	Endpoint	Purpose
GET	/api/users	Get all users
GET	/api/users/{id}	Get one user
POST	/api/users	Create user
PUT	/api/users/{id}	Update user
DELETE	/api/users/{id}	Delete user
GET	/api/users/{id}/posts	Get a user's posts

Posts

Method	Endpoint	Purpose
GET	/api/posts	Get all posts
GET	/api/posts/{id}	Get one post
POST	/api/posts	Create post
PUT	/api /posts/{id}	Update post
DELETE	/api /posts/{id}	Delete post
GET	/api /posts/{id}/interactions	Get interactions on a post
GET	/api/posts/{id}/author	Get a post with its author
GET	/api/posts?authorId={authorId}	Get posts filtered by author

Ayan Ghanchi

Interactions

Method	Endpoint	Purpose
GET	/api/interactions	Get all interactions
GET	/api /interactions/{id}	Get one interaction
POST	/api /interactions	Create interaction
PUT	/api /interactions/{id}	Update interaction
DELETE	/api /interactions/{id}	Delete interaction

Example Request and Response:

Create Post

Request

POST /posts


```
{
  "authorId": 1,
  "content": "This is my first post!",
  "mediaUrl": "https://example.com/image.jpg"
}
```


Response

201 Created


```
{
  "id": 10,
  "authorId": 1,
  "authorUsername": "agspeedz",
  "content": "This is my first post!",
  "mediaUrl": "https://example.com/image.jpg",
```

"createdAt": "2026-10-02T13:30:00"

}

Other expected status codes include:

- 200 OK for successful GET and PUT requests
- 201 Created for successful POST requests
- 204 No Content for successful DELETE requests
- 400 Bad Request for invalid input
- 404 Not Found when a resource does not exist

Validation:

User

- username: @NotBlank
- email: @NotBlank, @Email
- displayName: @NotBlank

Post

- authorId: @NotNull, @Positive
- content: @NotBlank
- mediaUrl: optional

Interaction

- userId: @NotNull, @Positive
- postId: @NotNull, @Positive
- type: @NotNull
- commentText: required when type is COMMENT

Invalid requests will return:

400 Bad Request

Ayan Ghanchi

Non-Functional Considerations

Error Handling

A NotFoundException will be used when a User, Post, or Interaction cannot be found.

Validation errors will be handled using MethodArgumentNotValidException.

Project Structure

The project uses presentation, service, datalayer, mapper, exception, bootstrap, and config packages.

The system separates Controller, Service, and Repository responsibilities.

Testing

The project includes Controller, Service, and Integration tests.

Tests verify successful CRUD operations, invalid input, nonexistent resources, and correct HTTP status codes.